

ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN - TIN

HUST

BÁO CÁO THỰC HÀNH
MÔN HỌC: TÍNH TÍNH SONG SONG

Giảng viên hướng dẫn: (Tên Giảng Viên)
Nhóm sinh viên thực hiện: Phạm Minh Thùy - MSHV: 20261051M
Trần Thị Khánh Huyền - MSHV: 20252875M

Hà Nội, Năm 2026

Mục lục

Giới thiệu về báo cáo	2
1 Bài tập 1: Cộng và Tích vô hướng hai Vector	3
1.1 Mô tả bài toán	3
1.2 Phân tích và Giải thích chi tiết mã nguồn	3
1.3 Kết quả thực nghiệm	5
2 Bài tập 2: Giải hệ phương trình tuyến tính	7
2.1 Lý thuyết phương pháp khử Gauss	7
2.2 Phân tích song song hóa bằng OpenMP	7
2.3 Kết quả thực nghiệm Gauss	9
2.4 Phương pháp lặp Jacobi (Khung đề mục hoàn thiện)	10
Kết luận	11

Giới thiệu về báo cáo

Môn học **Tính toán song song** cung cấp các kiến thức cơ bản và nâng cao về lập trình đa xử lý trên hệ thống máy tính hiện đại. Trong báo cáo này, nhóm thực hiện lập trình và thử nghiệm các bài toán số học cơ bản bằng công cụ **OpenMP** (Open Multi-Processing) - một API hỗ trợ lập trình song song bộ nhớ chia sẻ phổ biến và mạnh mẽ trên C/C++ và Fortran.

Nội dung báo cáo tập trung vào hai bài tập thực hành chính:

1. Bài tập 1: Cộng hai vector và mở rộng tính tích vô hướng của hai vector:

Thực hiện song song hóa vòng lặp tính toán trên vector, thử nghiệm hiệu năng và tính đúng đắn với các kích thước dữ liệu khác nhau.

2. Bài tập 2: Giải hệ phương trình tuyến tính: Sử dụng hai phương pháp phổ

biến là phương pháp khử Gauss và phương pháp lặp Jacobi. Báo cáo này sẽ trình bày chi tiết về phần thuật toán và lập trình song song của phương pháp khử Gauss.

Phần phương pháp Jacobi sẽ do thành viên khác trong nhóm phụ trách hoàn thiện.

Báo cáo bao gồm phân tích lý thuyết thuật toán, giải thích chi tiết từng đoạn mã nguồn C++ có tích hợp OpenMP và đưa ra các kết quả thử nghiệm thực tế từ chương trình.

Chương 1

Bài tập 1: Cộng và Tích vô hướng hai Vector

1.1 Mô tả bài toán

Cho hai vector số thực $A = (a_0, a_1, \dots, a_{n-1})$ và $B = (b_0, b_1, \dots, b_{n-1})$ cùng kích thước n .

- **Cộng hai vector:** Kết quả là một vector $C = A + B$ với các phần tử được tính theo công thức:

$$c_i = a_i + b_i \quad \forall i = 0, \dots, n-1 \quad (1.1)$$

- **Tích vô hướng hai vector:** Kết quả là một số thực $S = A \cdot B$ được tính theo công thức:

$$S = \sum_{i=0}^{n-1} a_i \times b_i \quad (1.2)$$

Cả hai bài toán đều có tính chất độc lập dữ liệu cao ở cấp độ phần tử, rất thích hợp cho việc lập trình song song hóa vòng lặp bằng cách chia các phần tử cho nhiều luồng (thread) xử lý đồng thời.

1.2 Phân tích và Giải thích chi tiết mã nguồn

Dưới đây là mã nguồn C++ tính tổng hai vector bằng OpenMP được cài đặt trong tệp `vector_sum_openmp.cpp`:

```
1 vector<double> sumVectors(const vector<double>& a, const vector<
  double>& b) {
2     // 1. Lay so chieu cua vector.
3     int n = static_cast<int>(a.size());
```

```

4 // 2. Tao vector ket qua co cung kich thuoc.
5 vector<double> result(n);
6
7 // 3. Tinh tong song song: moi luong xu ly mot phan phan tu.
8 #pragma omp parallel for
9 for (int i = 0; i < n; i++) {
10     result[i] = a[i] + b[i];
11 }
12
13 return result;
14 }

```

Listing 1.1: Hàm cộng hai vector song song bằng OpenMP

Giải thích chi tiết hoạt động của mã nguồn:

- **Dòng 3:** `int n = static_cast<int>(a.size());`
Lấy kích thước của vector đầu vào. Hàm trả về kiểu `size_t` nên ta dùng `static_cast<int>` để chuyển về kiểu số nguyên `int` nhằm đồng bộ kiểu dữ liệu với vòng lặp sau đó.
- **Dòng 5:** `vector<double> result(n);`
Cấp phát vùng nhớ cho vector kết quả có cùng kích thước n để chuẩn bị lưu giá trị cộng của từng phần tử.
- **Dòng 8:** `#pragma omp parallel for`
Đây là chỉ thị song song hóa vòng lặp quan trọng nhất của OpenMP.
 - `omp parallel`: Yêu cầu trình biên dịch tạo ra một nhóm các luồng (thread team) chạy song song.
 - `for`: Yêu cầu phân chia tự động các bước lặp của vòng lặp `for` ngay phía sau cho các luồng xử lý. Ví dụ, nếu CPU chạy với 4 luồng và $n = 1000$, luồng 0 sẽ xử lý chỉ số từ $0 \rightarrow 249$, luồng 1 từ $250 \rightarrow 499$, luồng 2 từ $500 \rightarrow 749$, và luồng 3 từ $750 \rightarrow 999$.
- **Dòng 9-10:** `for (int i = 0; i < n; i++) { result[i] = a[i] + b[i]; }`
Mỗi luồng thực hiện phép cộng trên đoạn chỉ số của riêng mình độc lập. Vì các luồng truy cập và ghi dữ liệu vào các phần tử khác nhau của mảng kết quả (`result[i]`), không xảy ra hiện tượng tranh chấp dữ liệu (Race Condition).

Mở rộng sang bài toán Tích vô hướng:

Khác với phép cộng vector (sinh ra một mảng mới), phép tính tích vô hướng yêu cầu dồn tất cả kết quả nhân phần tử $a_i \times b_i$ vào một biến vô hướng duy nhất S . Nếu song song hóa thông thường, nhiều luồng ghi đè vào cùng một địa chỉ bộ nhớ của biến S sẽ gây ra lỗi tranh chấp dữ liệu. Để giải quyết điều này, ta sử dụng cơ chế **Reduction** của OpenMP:

```

1 double dotProduct(const vector<double>& a, const vector<double>& b
  ) {
2     int n = static_cast<int>(a.size());
3     double sum = 0.0;
4
5     #pragma omp parallel for reduction(+:sum)
6     for (int i = 0; i < n; i++) {
7         sum += a[i] * b[i];
8     }
9     return sum;
10 }

```

Listing 1.2: Thuật toán tích vô hướng song song sử dụng reduction

Cơ chế hoạt động của reduction(+:sum):

1. OpenMP sẽ tạo ra bản sao cục bộ (private copy) của biến **sum** cho từng luồng và khởi tạo nó bằng giá trị trung hòa của phép cộng (là 0.0).
2. Mỗi luồng thực hiện cộng dồn tích $a_i \times b_i$ vào bản sao cục bộ của mình.
3. Sau khi vòng lặp kết thúc, các luồng tiến hành cộng gộp (reduce) an toàn các giá trị cục bộ này vào biến **sum** toàn cục để ra kết quả cuối cùng.

1.3 Kết quả thực nghiệm

Chương trình tính tổng vector đã được chạy kiểm nghiệm trên 18 bộ test có kích thước từ 3 đến 20. Dưới đây là bảng trích dẫn một số kết quả chạy tiêu biểu thu được:

Chương trình cho kết quả trùng khớp hoàn toàn với kết quả mong đợi trên toàn bộ các bộ test kích thước khác nhau, chứng minh tính đúng đắn của giải pháp song song.

Bảng 1.1: Kết quả chạy kiểm thử cộng hai vector song song

Bộ test thứ	Kích thước (n)	Vector đầu vào (A và B)	Kết quả ($C = A + B$)
1	3	$A = [1, 2, 3], B = [4, 5, 6]$	$[5, 7, 9]$
2	4	$A = [1..4], B = [5..8]$	$[6, 8, 10, 12]$
3	5	$A = [1..5], B = [6..10]$	$[7, 9, 11, 13, 15]$
4	10	$A = [1..10], B = [11..20]$	$[12, 14, 16, 18, 20, 22, 24, 26, 28, 30]$
5	20	$A = [1..20], B = [21..40]$	$[22, 24, 26, \dots, 58, 60]$

Chương 2

Bài tập 2: Giải hệ phương trình tuyến tính

2.1 Lý thuyết phương pháp khử Gauss

Hệ phương trình tuyến tính có dạng $Ax = B$, trong đó A là ma trận hệ số kích thước $n \times n$, B là vector vế phải kích thước n , và x là vector ẩn cần tìm.

Phương pháp khử Gauss chia làm hai giai đoạn chính:

1. **Khử xuôi (Forward Elimination)**: Biến đổi ma trận bổ sung $[A|B]$ về dạng ma trận tam giác trên thông qua các phép biến đổi sơ cấp dòng.
2. **Thế ngược (Backward Substitution)**: Tính toán lần lượt giá trị của các nghiệm từ nghiệm cuối cùng x_{n-1} lên đến nghiệm đầu tiên x_0 .

2.2 Phân tích song song hóa bằng OpenMP

Trong giai đoạn Khử xuôi, tại mỗi bước lặp cột i (chốt), ta cần biến đổi tất cả các dòng nằm phía dưới dòng i . Các dòng này có thể được xử lý độc lập với nhau, đây là cơ hội tuyệt vời để song song hóa.

Dưới đây là phần triển khai mã nguồn giải hệ phương trình tuyến tính bằng phương pháp khử Gauss trong tệp `solvers.cpp`:

```
1 std::vector<double> solveGauss(const std::vector<std::vector<  
    double>>& A, const std::vector<double>& B) {  
2     int n = A.size();  
3     std::vector<std::vector<double>> tempA = A;  
4     std::vector<double> tempB = B;  
5
```



```

6   for (int i = 0; i < n; ++i) {
7       // 1. Chọn phần tử trội (partial pivoting)
8       int pivot = i;
9       for (int j = i + 1; j < n; ++j) {
10          if (std::abs(tempA[j][i]) > std::abs(tempA[pivot][i]))
11      {
12          pivot = j;
13      }
14      if (pivot != i) {
15          std::swap(tempA[i], tempA[pivot]);
16          std::swap(tempB[i], tempB[pivot]);
17      }
18      if (std::abs(tempA[i][i]) < 1e-9) {
19          throw std::runtime_error("Ma tran suy bien, không thể
giai bằng Gauss.");
20      }
21
22      // 2. Khu xúi song song hóa bằng OpenMP
23      #pragma omp parallel for if(n >= 3)
24      for (int j = i + 1; j < n; ++j) {
25          double factor = tempA[j][i] / tempA[i][i];
26          tempA[j][i] = 0.0;
27          for (int k = i + 1; k < n; ++k) {
28              tempA[j][k] -= factor * tempA[i][k];
29          }
30          tempB[j] -= factor * tempB[i];
31      }
32  }
33
34  // 3. Thế ngược (tuan tu vi có tính phụ thuộc dữ liệu ngược)
35  std::vector<double> x(n);
36  for (int i = n - 1; i >= 0; --i) {
37      double sum = 0.0;
38      for (int j = i + 1; j < n; ++j) {
39          sum += tempA[i][j] * x[j];
40      }
41      x[i] = (tempB[i] - sum) / tempA[i][i];
42  }
43  return x;

```

Listing 2.1: Thuật toán khử Gauss song song hóa với OpenMP

Phân tích chi tiết mã nguồn:

- **Phần 1: Chọn phần tử trội (Dòng 7-20)**

Trong quá trình tính toán số học, để giảm thiểu sai số làm tròn và tránh lỗi chia cho 0, việc tìm dòng chứa phần tử có giá trị tuyệt đối lớn nhất ở cột đang xét để đưa lên làm phần tử chốt (pivot) là cực kỳ quan trọng. Quá trình chọn phần tử trội này được thực hiện tuần tự để đảm bảo tính chính xác trước khi thực hiện khử.

- **Phần 2: Khử xuôi song song hóa (Dòng 22-31)**

Chỉ thị `#pragma omp parallel for if(n >= 3)` được sử dụng.

- Mệnh đề `if(n >= 3)` giúp kiểm tra điều kiện kích thước ma trận. Với các hệ phương trình quá nhỏ (ví dụ 2×2), chi phí tạo lập và đồng bộ hóa luồng (thread overhead) lớn hơn thời gian tính toán thực tế. Vì vậy, ta chỉ song song hóa khi ma trận đủ lớn ($n \geq 3$).
- Mỗi luồng phụ trách thực hiện trừ dòng dòng j đi một tỷ lệ thích hợp so với dòng chốt i để triệt tiêu hệ số tại cột i . Do các phép biến đổi trên các dòng j khác nhau là độc lập và không ghi đè lẫn nhau, quá trình này hoàn toàn an toàn và đạt gia tốc cao trên phần cứng đa nhân.

- **Phần 3: Thế ngược tuần tự (Dòng 34-43)**

Giai đoạn thế ngược không thể song song hóa theo cách đơn giản vì bước tính nghiệm x_i phụ thuộc trực tiếp vào các nghiệm đã tính trước đó $x_{i+1}, x_{i+2}, \dots, x_{n-1}$. Do đó, vòng lặp thế ngược chạy lùi từ $n - 1$ về 0 bắt buộc phải thực hiện tuần tự để bảo toàn tính đúng đắn về mặt toán học.

2.3 Kết quả thực nghiệm Gauss

Hàm giải hệ phương trình tuyến tính bằng thuật toán khử Gauss song song đã được chạy kiểm nghiệm thực tế trên bộ dữ liệu kiểm thử kích thước 3×3 trong `input.json`:

- **Đầu vào:**

$$A = \begin{bmatrix} 3 & 2 & -4 \\ 2 & 3 & 3 \\ 5 & -3 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 3 \\ 15 \\ 14 \end{bmatrix}$$

- **Kết quả chạy chương trình:**

- Kết quả nghiệm tìm được: $x = [3, 1, 2]$.
- Nghiệm kỳ vọng mong đợi: $x_{exp} = [3, 1, 2]$.
- Sai số tối đa thu được so với nghiệm chuẩn: 4.44089×10^{-16} .
- Trạng thái: **PASSED** (Đạt chuẩn chính xác cao).

2.4 Phương pháp lặp Jacobi (Khung đề mục hoàn thiện)

Lưu ý: Phần phương pháp lặp Jacobi và song song hóa của nó do thành viên Trần Thị Khánh Huyền chịu trách nhiệm nghiên cứu, cài đặt mã nguồn và sẽ trình bày chi tiết tại mục này trong các phiên bản báo cáo tiếp theo.

Kết luận

Qua bài thực hành lớn môn học Tính toán song song, nhóm đã thu hoạch được các kết quả quan trọng sau:

1. Hiểu rõ và áp dụng thành công API OpenMP để tăng tốc các bài toán tính toán số học phổ biến bao gồm các phép toán trên vector (cộng, tích vô hướng) và giải hệ phương trình đại số tuyến tính (phương pháp khử Gauss).
2. Nắm vững các chỉ thị song song cơ bản như `#pragma omp parallel for`, kỹ thuật xử lý tranh chấp dữ liệu bằng `reduction`, và cách tinh chỉnh hiệu năng thông qua mệnh đề điều kiện `if`.
3. Kiểm chứng tính đúng đắn và độ chính xác của các chương trình đã song song hóa thông qua các bộ dữ liệu thử nghiệm thực tế với nhiều kích thước khác nhau.

Nhóm sinh viên chân thành cảm ơn sự hướng dẫn tận tình của thầy cô bộ môn giúp nhóm hoàn thành tốt bài thực hành này.